

InterviewPal — مستند پیاده‌سازی فرانت‌اند

اپلیکیشن تمرین مصاحبه فنی برای برنامه‌نویسان فارسی‌زبان

مورد	مقدار
فناوری	TypeScript 6 و Angular 22 (standalone, zoneless, signals)
ریپو	github.com/nasibehash/interviewPal-frontend
اپ زنده	interview-pal-frontend-sable.vercel.app
API	interviewpal-backend.onrender.com (ASP.NET Core، ریپوی interviewPal-backend)
وضعیت	فاز ۲: استفاده از برنامه فقط با حساب کاربری (ثبت‌نام و ورود) ممکن است و پیشرفت روی سرور ذخیره می‌شود

۱. هدف و دامنه محصول

InterviewPal به برنامه‌نویس کمک می‌کند برای مصاحبه فنی تمرین کند. محتوا فارسی است و اصطلاح‌های فنی و کدها انگلیسی می‌مانند. فرانت‌اند دو بخش اصلی دارد:

- تمرین مصاحبه: ۳۰۰ سؤال در شش فناوری (Angular، JavaScript، TypeScript، React، Next.js و NET) در سه سطح جونیور، مید و سنیور و سه حالت تمرین: یادگیری، مصاحبه (با تایمر) و فلش‌کارت.
- الگوریتم و الگوهای طراحی: ۲۴ درس (۱۴ الگوریتم و ۱۰ الگوی طراحی) که هر کدام مثال واقعی دارد و کدش به زبان یا فریم‌ورک انتخابی کاربر نمایش داده می‌شود، همراه با تمرین.
- پیشرفت من: تاریخچه تمرین‌ها، میانگین نمره، روزهای پشت‌سرهم و سؤال‌های ضعیف.
- حساب کاربری: ثبت‌نام و ورود با ایمیل و رمز عبور. با ورود، تاریخچه و پیشرفت درس‌ها روی سرور ذخیره می‌شود و در هر دستگاه در دسترس است؛ داده‌های قبلی همان مرورگر (از نسخه‌های بدون حساب) هنگام اولین ورود منتقل می‌شوند. بدون ورود فقط صفحه‌های ورود و ثبت‌نام باز می‌شوند.
- ریسپانسیو: طرح از عرض ۳۶۰ پیکسل (موبایل) تا دسکتاپ بدون اسکرول افقی کار می‌کند؛ منو در موبایل ردیف جدا و قابل اسکرول دارد.

۲. پشته فناوری و تصمیم‌های اصلی

حوزه	انتخاب	دلیل
فریم‌ورک	Angular 22، کامپوننت‌های standalone	آخرین نسخه؛ بدون NgModule و با بارگذاری تنبل ساده
تشخیص تغییر	Zoneless و OnPush (هر دو پیش‌فرض Angular 22)	بدون zone.js؛ همه state با signal است و فقط بخش تغییرکرده بررسی می‌شود
state	signal و computed در سرویس‌های root	بدون کتابخانه state جدا؛ جریان داده ساده و قابل‌تست

حوزه	انتخاب	دلیل
داده از API	<code>httpResource</code> (داخل <code>ApiClient</code>) برای خواندن، <code>firstValueFrom</code> برای نوشتن	خواندن‌ها reactive اند و با تغییر signal دوباره درخواست می‌شوند
فرم‌ها	Signal Forms (<code>@angular/forms/signals</code>)	مدل فرم یک signal ساده است؛ بدون <code>FormsModule</code> و <code>ngModel</code>
قالب	کنترل‌فلو <code>@empty</code> / <code>@for</code> / <code>@if</code>	سینتکس جدید Angular، بدون <code>*ngIf</code>
زبان و جهت	فارسی، <code>dir="rtl"</code> و خصوصیت‌های منطقی CSS	یک پایه برای RTL؛ کدها جداگانه LTR نمایش داده می‌شوند
فونت	Vazirmatn (بسته <code>@fontsource-variable/vazirmatn</code>)	بدون درخواست به CDN بیرونی
Markdown	<code>marked</code> و <code>DOMPurify</code> و <code>highlight.js</code>	محتوای درس‌ها Markdown است؛ بعد از تبدیل پاک‌سازی (sanitize) می‌شود
احراز هویت	JWT کوتاه‌عمر در حافظه و <code>refresh token</code> در کوکی <code>httpOnly</code>	توکن دسترسی هیچ‌جا ذخیره نمی‌شود و جاوااسکریپت به <code>refresh token</code> دسترسی ندارد (جلوگیری از سرقت با XSS)
تست	Vitest از طریق <code>ng test</code>	اجرای سریع و ساده؛ ۸۷ تست واحد
استایل	SCSS ساده با متغیرهای CSS و تم روشن/تیره	بدون کتابخانه UI؛ حجم کم

ذخیره‌سازی بدون حساب و با حساب

کاربر مهمان: تاریخچه و پیشرفت در `localStorage` و تمرین نیمه‌کاره در `sessionStorage` می‌ماند. همه خواندن و نوشتن‌ها از یک لایه امن (safe-storage) می‌گذرند تا در حالت مرور خصوصی یا ذخیره‌سازی مسدود، برنامه نشکند. کاربر واردشده: `ProgressStore` و `LessonProgress` همان API قبلی را نگه می‌دارند ولی منبع داده سرور است؛ پس صفحه‌ها تغییری نکردند (بخش ۵.۴).

```
src/app/
  app.ts / app.html / app.scss  shell: header, nav, router-outlet
  app.config.ts                 providers: http + interceptors, router features, session
restore
  app.routes.ts                 lazy routes and guards
  core/                         logic and data, no UI
    models.ts                   TypeScript types of the API contract
    api-client.ts               the only place that talks HTTP (resources + calls)
    api-error.ts                interceptor: HTTP failure -> ApiError (Persian message)
    auth-store.ts               user, access token (memory), login/register/logout/restore
    auth-interceptor.ts         adds the bearer token, refreshes once on 401
    auth-guards.ts              authGuard / guestGuard
    practice-session.ts         state of the practice in progress
    progress-store.ts           history and stats (localStorage or server)
    lesson-progress.ts          progress of lesson exercises (localStorage or server)
    preferred-technology.ts     the language the learner picked
    self-assessment.ts          "I knew it" -> a gradable answer
    safe-storage.ts             storage reads/writes that never throw
  shared/                       shared components (ts + html + scss each)
    markdown.ts, markdown.pipe.ts, question-view, answer-panel,
    report-dialog, code-block, field-errors
  features/                     pages, each one lazy loaded
    setup/ practice/ result/ history/ lessons/ auth/ account/
docs/
  frontend.md                   this document (source)
  InterviewPal-Frontend.pdf     the same document as PDF
```

در این ساختار، هر پوشه یک مسئولیت دارد: **core** منطق و داده، **shared** اجزای تکراری و **features** صفحه‌ها.

قرارداد نام‌گذاری فایل‌ها

از Angular 20 به بعد راهنمای رسمی سبک (style guide) و دستور **ng generate** پسوند **.component** / **.service** را حذف کرده‌اند. به جای **lesson.component.ts** نام فایل **lesson.ts** و نام کلاس **Lesson** است (نه **LessonComponent**). این پروژه همین قرارداد را دنبال می‌کند:

- هر کامپوننت سه فایل جدا دارد: **name.ts**، **name.html** و **name.scss** (با **templateUrl** و **styleUrl**). قالب و استایل inline فقط در تست‌ها استفاده می‌شود.
- نام صفحه‌ها پسوند **-page** دارد (**lesson-page.ts**) تا از کامپوننت‌های کوچک‌تر در پوشه **shared** جدا شوند؛ این یک انتخاب پروژه است، نه الزام Angular.
- سرویس‌ها بدون پسوند و با نام معنادار: **ApiClient**، **ProgressStore**.
- قاعده اصلی: **core** به هیچ کامپوننتی وابسته نیست و **features** فقط از **core** و **shared** استفاده می‌کنند. هر صفحه با **LoadComponent** بارگذاری می‌شود و bundle اولیه کوچک می‌ماند (حدود ۹۲ کیلوبایت فشرده).

۴. مسیریابی

مسیر	صفحه	توضیح
/	شروع تمرین	انتخاب تکنولوژی، سطح، تعداد (۵ تا ۱۰۰) و حالت؛ همه مسیرها به جز <code>/login</code> و <code>/register</code> پشت <code>AuthGuard</code> هستند
/practice	تمرین	guard: فقط وقتی تمرین فعال وجود دارد
/result	نتیجه	guard: فقط بعد از تمام شدن و ارزیابی
/lessons	فهرست درسها	تب الگوریتمها و الگوهای طراحی، فیلتر سطح
/lessons/:id	درس	پارامتر <code>id</code> با <code>withComponentInputBinding</code> مستقیم به <code>input()</code> می‌رسد
/history	پیشرفت من	آمار و تاریخچه
/login	ورود	<code>guestGuard</code> : کاربر وارد شده به <code>/</code> یا مسیر <code>redirect</code> می‌رود؛ مهمان بعد از ورود به همان صفحه خواسته شده برمی‌گردد
/register	ثبت نام	فقط مهمان؛ بعد از موفقیت مستقیم وارد می‌شود
/account	حساب من	guard: فقط کاربر وارد شده؛ مهمان به <code>/login?redirect=/account</code> می‌رود
**	—	بازگشت به صفحه اصلی

guardها تابعی (`CanActivateFn`) هستند. guardهای تمرین فقط وضعیت `PracticeSessionStore` را می‌خوانند: اگر کاربر مستقیم `/practice` را باز کند و تمرینی نداشته باشد، به `/` هدایت می‌شود. `AuthGuard` و `guestGuard` اول منتظر `auth.ready` (پایان بازیابی نشست) می‌مانند تا با `refresh` صفحه کاربر وارد شده به اشتباه به `/login` نرود.

۵. لایه core

۵.۱ مدلها و ApiClient

- `models.ts` آیینة قرارداد JSON بک اند است (`camelCase` و `enum` به صورت رشته). سؤالها بدون جواب می‌آیند و جواب فقط بعد از پاسخ دادن از API گرفته می‌شود، پس جواب در `payload` اولیه نیست. `ApiClient` تنها جایی است که `HttpClient` را می‌شناسد و آدرس پایه از توکن `API_BASE_URL` (پیش فرض `/api`) می‌آید. دو دسته متد دارد:
- خواندن به صورت `resource`: `technologiesResource()`، `lessonsResource()` و `lessonResource(id)` (برمی گردانند. صفحه‌ها آن را در `field initializer` می‌سازند و فقط `value()`، `isLoading()`، `error()` و `reload()` را می‌بینند؛ هیچ صفحه‌ای URL نمی‌سازد. `lessonResource` دو تابع (برای `id` و زبان) می‌گیرد و با تغییر هر کدام خودش دوباره درخواست می‌زند.
 - نوشتن و بقیه به صورت `Observable`: ارزیابی پاسخ، ساخت تمرین و گزارش، که در state با `firstValueFrom` مصرف می‌شوند.

`apiErrorInterceptor` (فایل `api-error.ts`) هر خطای HTTP را به `ApiError` با پیام فارسی تبدیل می‌کند: • (شبکه)، ۴۰۰ (متن `ProblemDetails` سرور)، ۴۰۴، ۴۲۹ و ۵xx. بنابراین `store` ها و صفحه‌ها هیچ‌وقت `HttpErrorResponse` خام نمی‌بینند.

۵.۲ PracticeSessionStore

قلب بخش تمرین است و کل `state` را با `signal` نگه می‌دارد: `session`، `answers`، `revealed`، `correctness`، `notes`، `index`، `startedAt`، `phase`، `evaluation`، `busy` و `error`. فاز تمرین یکی از مقدارهای زیر است:

```
idle -> practicing -> (self-assessment) -> submitting -> finished
      ^                               |
      +----- evaluation failed -----+
```

- حالت یادگیری: با انتخاب هر گزینه، پاسخ فوراً به `API` (`check`) فرستاده می‌شود و درست/غلط و توضیح نشان داده می‌شود؛ پاسخ قفل می‌شود.
- حالت مصاحبه: پاسخ‌ها فقط در حافظه جمع می‌شوند و در پایان با یک درخواست (`evaluate`) ارزیابی می‌شوند. سؤال‌های تشریحی که کاربر خودارزیابی نکرده، قبل از ارسال وارد فاز `self-assessment` می‌شوند.
- حالت فلش‌کارت: جواب نمایش داده می‌شود و کاربر خودش «بلد بودم / بلد نبودم» می‌زند.
- سؤال بی‌جواب در انتها غلط حساب می‌شود (`completeAnswers`).
- تمرین نیمه‌کاره در `sessionStorage` ذخیره می‌شود و با `refresh` از همان سؤال ادامه پیدا می‌کند (با `effect`).
- اگر ارزیابی با خطا روبه‌رو شود، فاز به `practicing` برمی‌گردد تا کاربر دوباره تلاش کند.

تابع `answerFromSelfAssessment` نتیجه خودارزیابی را به پاسخی تبدیل می‌کند که `API` بفهمد: برای سؤال تشریحی `knewIt` و برای سؤال گزینه‌ای، گزینه درست (اگر کاربر بلد بوده) یا یک گزینه غلط.

۵.۳ ProgressStore و دیگر سرویس‌ها

سرویس	کلید storage	کار
ProgressStore	<code>interviewpal.history.v1</code> و <code>interviewpal.question-stats.v1</code>	حداکثر ۱۰۰ تمرین اخیر؛ آمار هر سؤال؛ میانگین، streak و سؤال‌های ضعیف (computed)
PracticeSessionStore	<code>interviewpal.active-session.v1</code> (sessionStorage)	تمرین در حال انجام
LessonProgress	<code>interviewpal.lesson-progress.v2</code>	پاسخ آخر هر تمرین درس؛ درس وقتی کامل است که همه تمرین‌ها پاسخ داده و آخرین پاسخ‌ها درست باشند
PreferredTechnology	<code>interviewpal.technology.v1</code>	زبان انتخابی؛ مقدار نامعتبر به <code>javascript</code> برمی‌گردد

نسخه (`.v1`، `.v2`) در نام کلیدها گذاشته شده تا اگر ساختار داده عوض شد، داده‌های قدیمی بی‌صدا نادیده گرفته شوند و برنامه نشکند.

۵.۴ حساب کاربری و همگام‌سازی

AuthStore وضعیت کاربر را با signal نگه می‌دارد: `user`، `isLoggedIn` و `ready`. توکن دسترسی (JWT، ۱۵ دقیقه) فقط در حافظهٔ همین سرویس است و در storage نوشته نمی‌شود. refresh token را سرور در کوکی `httpOnly` و `SameSite=Strict` با مسیر `/api/auth` می‌گذارد و جاوااسکریپت آن را نمی‌بیند.

- **بازبایی نشست:** `provideAppInitializer` متد `restore()` را صدا می‌زند. فقط اگر قبلاً نشانهٔ ورود (یک پرچم غیرحساس در storage) ثبت شده باشد، `POST /api/auth/refresh` فرستاده می‌شود؛ مهمان‌ها درخواست اضافه نمی‌فرستند.
- **authInterceptor:** روی درخواست‌های `/api`، هدر `Authorization: Bearer` اضافه می‌کند. با پاسخ ۴۰۱ یک بار refresh می‌زند (چند درخواست هم‌زمان یک refresh مشترک دارند) و درخواست را تکرار می‌کند؛ اگر refresh رد شد، کاربر خارج می‌شود. ترتیب interceptorها `[apiErrorInterceptor, authInterceptor]` است تا خطای ۴۰۱ خام به interceptor احرار هویت برسد.
- **ورود و ثبت‌نام:** فرم‌ها Signal Forms اند. در ثبت‌نام، تکرار رمز با `validate` و `valueOf` به فیلد رمز مقایسه می‌شود (اعتبارسنجی بین‌فیلدی). خطاهای فیلد را کامپوننت مشترک `FieldErrors` نشان می‌دهد.
- **حساب من:** تغییر رمز (سرور همهٔ نشست‌ها را می‌بندد و کاربر با رمز جدید دوباره وارد می‌شود)، حذف حساب با تأیید رمز و حذف پیشرفت ذخیره‌شده روی سرور.

همگام‌سازی پیشرفت: `ProgressStore` و `LessonProgress` با یک `effect` روی `auth.user()` بین دو منبع جابه‌جا می‌شوند.

وضعیت	منبع داده	رفتار
مهمان	—	به صفحه‌ای جز ورود و ثبت‌نام دسترسی ندارد؛ <code>localStorage</code> فقط بقایای نسخه‌های قبلی را نگه می‌دارد
ورود	سرور (<code>/api/me/...</code>)	با ورود، تاریخچه و پیشرفت درس‌ها بارگذاری می‌شود؛ نتیجهٔ تمرین بعدی را خود سرور هنگام ارزیابی ثبت می‌کند
اولین ورود	<code>POST /api/me/progress/import</code>	داده‌های محلی یک‌بار منتقل می‌شوند و بعد از موفقیت از مرورگر پاک می‌شوند
خروج	<code>localStorage</code>	دوباره حالت مهمان؛ داده‌های حساب قبلی در مرورگر نمی‌ماند، چون بعد از ورود از سرور خوانده می‌شود و نسخهٔ محلی بعد از انتقال پاک شده است

بک‌اند هم همین قاعده را اعمال می‌کند: مسیرهای سؤال، درس و تمرین بدون توکن 401 می‌دهند، پس دور زدن رابط کاربری ممکن نیست. اگر نشست وسط کار تمام شود (خروج، refresh ردشده، حذف حساب)، App کاربر را به `/login` می‌برد.

۶. صفحه‌ها

۶.۱ شروع تمرین (setup)

فهرست تکنولوژی‌ها با `httpResource` از `/technologies` می‌آید و حالت‌های loading و خطا (با دکمهٔ «تلاش دوباره») دارد. انتخاب تکنولوژی و سطح چندانگانه است و خالی بودن یعنی «همه». تعداد سؤال با اسلایدر ۵ تا ۱۰۰ و زمان تقریبی (حدود

۸۰ ثانیه برای هر سؤال) نشان داده می‌شود.

۶.۲ تمرین (practice)

- نوار پیشرفت و شمارنده «سؤال n از N»؛ دکمه‌های قبلی/بعدی و «پایان تمرین» در سؤال آخر.
- در حالت مصاحبه تایمر معکوس از `timeLimitSeconds` نشان داده می‌شود؛ زیر یک دقیقه قرمز می‌شود و با صفر شدن، تمرین خودکار ارسال می‌شود.
- گزینه‌ها بعد از پاسخ به رنگ درست (سبز) و غلط (قرمز) درمی‌آیند و پاسخ کامل (جواب کوتاه، توضیح، اشتباه رایج و سؤال بعدی مصاحبه‌کننده) نمایش داده می‌شود.
- گزارش مشکل یک سؤال (`report-dialog`) بعد از دیدن جواب در دسترس است.

۶.۳ نتیجه (result)

درصد و تعداد پاسخ درست، نمودار میله‌ای به تفکیک تکنولوژی و سطح، موضوع‌های ضعیف (tag) و مرور همه سؤال‌ها با توضیح. دکمه «تمرین دوباره با سؤال‌های ضعیف» با `startFromQuestions` یک تمرین یادگیری از همان سؤال‌ها می‌سازد.

۶.۴ پیشرفت من (history)

تعداد تمرین، میانگین نمره، روزهای پشت‌سرهم، دقت هر تکنولوژی، فهرست سؤال‌های ضعیف و ۱۵ تمرین آخر. دکمه پاک‌کردن سابقه بعد از تأیید کاربر همه داده‌ها را حذف می‌کند.

۶.۵ درس‌ها (lessons)

- فهرست: انتخاب زبان در بالای صفحه، دو تب (الگوریتم‌ها / الگوهای طراحی)، فیلتر سطح، گروه‌بندی بر اساس دسته و نشان ✓ کامل» روی درس‌های تمام‌شده.
- جست‌وجو: یک Signal Form با `debounce` ۲۵۰ میلی‌ثانیه؛ در عنوان، خلاصه و تگ‌ها می‌گردد و حروف عربی و فارسی («ك»/«ک» و «ي»/«ی») را یکی می‌گیرد.
- صفحه درس: مثال واقعی، توضیح، پیچیدگی (برای الگوریتم‌ها)، کد به زبان انتخابی با دکمه کپی، شرح کد، «کی استفاده کنم / کی نه»، اشتباه رایج و ۴ تمرین.
- عوض کردن زبان در وسط درس، درس را با `technology` جدید دوباره از API می‌گیرد؛ چون `httpResource` به signal زبان وابسته است.
- بخش تمرین‌ها داخل `@defer (on viewport)` است: کد آن (کامپوننت `LessonExerciseView`) در یک chunk جدا بارگذاری می‌شود و فقط وقتی کاربر به پایین صفحه برسد.
- هر تمرین جدا با API بررسی می‌شود (جواب درست در payload درس نیست) و نتیجه در `LessonProgress` ثبت می‌شود.

۷. کامپوننت‌های مشترک

کامپوننت	کار
markdown.ts و MarkdownPipe	تبدیل Markdown به HTML، رنگ‌آمیزی کد با <code>highlight.js</code> (فقط زبان‌های لازم ثبت شده‌اند: JavaScript، TypeScript، HTML، C#، JSON، bash، CSS و فقط بعد از آن <code>DOMPurify</code> و <code>bypassSecurityTrustHtml</code>)
QuestionView	متن سؤال، سطح (رنگی)، تگ‌ها و قطعه کد (همیشه <code>dir="ltr"</code>)
AnswerPanel	جواب کوتاه، توضیح کامل، اشتباه رایج و سؤال بعدی
ReportDialog	گزارش جواب غلط، قدیمی، مبهم یا غلط نگارشی؛ با <code>Signal Form</code> ، <code>required</code> ، <code>maxLength(500)</code> و <code><form [formRoot]></code>
CodeBlock	نمایش کد با زبان و دکمه «کپی کد» (clipboard)؛ خروجی <code>highlight.js</code> خودش کد را escape می‌کند

امنیت

همه متن‌های Markdown (سؤال، توضیح، درس) قبل از رسیدن به DOM با `DOMPurify` پاک‌سازی می‌شوند. اسکرپت و `onerror` حذف می‌شود و این رفتار تست واحد دارد. هیچ `innerHTML` دیگری بدون این مسیر استفاده نشده است.

۸. ویژگی‌های Angular 22 که در پروژه استفاده شده

ویژگی	کجا	چرا
Zoneless (پیش‌فرض)	<code>bootstrapApplication</code> بدون <code>zone.js</code>	رندر فقط با تغییر <code>signal</code> ؛ بدون <code>patch</code> کردن API‌های مرورگر
<code>OnPush</code> پیش‌فرض	همه کامپوننت‌ها (بدون تنظیم صریح)	در Angular 22 پیش‌فرض است؛ همه <code>state</code> با <code>signal</code> است، پس سازگار است
<code>httpResource</code> (پایدار از 22.0)	<code>ApiClient.technologiesResource()</code> ، <code>lessonsResource()</code> ، <code>lessonResource()</code>	خواندن <code>reactive</code> : با تغییر <code>id</code> یا زبان خودش دوباره درخواست می‌زند و <code>reload / error / isLoading</code> می‌دهد؛ <code>default value: []</code> الگوی <code>?? []</code> را از قالب‌ها حذف کرد
Signal Forms (پایدار از 22.0)	<code>ReportDialog</code> و جست‌وجوی <code>LessonsPage</code>	<code><form</code> ، <code>[formField]</code> ، <code>form()</code> ، <code>[formRoot]></code> با <code>submission</code> و <code>maxLength</code> ، <code>required</code> ، <code>debounce</code> ؛ مدل فرم یک <code>signal</code> ساده است
<code>linkedSignal</code>	<code>LessonExerciseView</code>	<code>state</code> قابل‌نوشتن که با عوض شدن <code>exercise</code> ورودی خودش بازنشانی می‌شود

ویژگی	کجا	چرا
@defer (on viewport)	تمرین‌های LessonPage	کد تمرین‌ها در یک chunk جدا و فقط با رسیدن به پایین صفحه بارگذاری می‌شود
@let	LessonPage	متغیر محلی در قالب، به جای تکرار technology.slug()
کنترل‌فلو جدید	همه قالب‌ها	@if ، @for با track ، @empty
/ input() / input.required()	کامپوننت‌ها و LessonPage	ورودی‌های signal؛ با withComponentInputBinding پارامتر :id مسیر مستقیم به input می‌رسد
computed و effect	storeها و صفحه‌ها	مشتق‌های خالص با computed ؛ effect فقط برای هماهنگ‌سازی با sessionStorage و تایمر
Functional interceptor	apiErrorInterceptor و authInterceptor با withInterceptors	تبدیل خطاها به ApiError و افزودن/تازه‌سازی توکن در یک نقطه
Router	withViewTransitions ، withInMemoryScrolling ، loadComponent ، تابعی	انتقال انیمیشنی بین صفحه‌ها، برگرداندن اسکرول و bundle اولیه کوچک
provideAppInitializer	app.config.ts	بازیابی نشست قبل از اولین رندر؛ guard با auth.ready منتظر همین کار می‌ماند
Signal Forms :validate و valueOf	AccountPage ، RegisterPage	اعتبارسنجی بین‌فیلدی (تکرار رمز) بدون کد دستی؛ FieldErrors پیام‌ها را از field.errors() می‌خواند
inject()	همه‌جا	بدون constructor injection؛ کار با field initializer و تابع‌های کمکی ساده می‌شود
Vitest با ng test	همه تست‌ها	@angular/build:unit-builder ، test؛ بدون Karma و Jasmine

بعضی از این‌ها، مثل httpResource و Signal Forms، در نسخه‌های قبلی آزمایشی بودند. در Angular 22 هر دو در تایپ‌های خود فریم‌ورک با @publicApi 22.0 (یعنی API عمومی و پایدار) علامت خورده‌اند.

۹. استایل، RTL و دسترس‌پذیری

- متغیرهای CSS در root: و تم تیره با prefers-color-scheme؛ رنگ اصلی بنفش (#4F46E5) و رنگ‌های جدا برای موفقیت، خطا و هشدار.

- خصوصیت‌های منطقی (`inset-block-start` ، `inline-size` ، `margin-inline`) به‌جای `left/right` تا جهت RTL خودکار درست شود.
- کد و عبارت‌های انگلیسی `dir="ltr"` و `unicode-bidi: plaintext` دارند تا در متن فارسی به‌هم نریزند.
- دکمه‌های انتخابی `aria-pressed` ، تب‌ها `role="tab"` و `aria-selected` ، تایمر `role="timer"` ، خطاها `role="alert"` و نوار پیشرفت `role="progressbar"` دارند. `focus` قابل‌مشاهده است.
- `prefers-reduced-motion` انیمیشن‌ها را خاموش می‌کند و طرح روی عرض موبایل بدون اسکرول افقی کار می‌کند.

۱۰. قرارداد API

متد	مسیر	استفاده
GET	<code>/api/technologies</code>	فهرست تکنولوژی‌ها و تعداد سؤال هر سطح
POST	<code>/api/practice/sessions</code>	ساخت تمرین (تکنولوژی‌ها، سطح‌ها، تعداد، حالت)
POST	<code>/api/practice/questions/{id}/check</code>	ارزیابی یک پاسخ (یادگیری)
POST	<code>/api/practice/evaluate</code>	ارزیابی کل تمرین (مصاحبه)
GET	<code>/api/questions/{id}</code>	یک سؤال با جواب (برای تمرین دوباره و فلش‌کارت)
POST	<code>/api/questions/{id}/reports</code>	گزارش مشکل سؤال
GET	<code>/api/lessons?kind=</code>	فهرست درس‌ها
GET	<code>/api/lessons/{id}?technology=</code>	درس با کد زبان انتخابی
POST	<code>/api/lessons/{id}/exercises/{exerciseId}/check</code>	بررسی یک تمرین درس (با ورود، نتیجه ذخیره می‌شود)
POST	<code>/api/auth/register</code>	ثبت‌نام؛ توکن دسترسی در بدنه و <code>refresh token</code> در کوکی
POST	<code>/api/auth/login</code>	ورود (قفل موقت بعد از چند رمز اشتباه و محدودیت تعداد درخواست)
POST	<code>/api/auth/refresh</code>	توکن دسترسی جدید و چرخش <code>refresh token</code>
POST	<code>/api/auth/logout</code>	پایان نشست و پاک‌کردن کوکی
GET	<code>/api/auth/me</code>	اطلاعات کاربر فعلی
POST	<code>/api/auth/change-password</code>	تغییر رمز؛ همه نشست‌ها بسته می‌شوند
POST	<code>/api/auth/delete-account</code>	حذف حساب و داده آن با تأیید رمز
GET	<code>/api/me/progress</code>	تاریخچه، آمار سؤال‌ها و پیشرفت درس‌های کاربر
POST	<code>/api/me/progress/import</code>	انتقال یک‌باره داده محلی به حساب

متد	مسیر	استفاده
DELETE	/api/me/progress	پاک کردن پیشرفت ذخیره شده

مسیرهای `/api/me/*` و تغییر رمز و حذف حساب نیاز به هدر `Authorization` دارند. کوکی `refresh` فقط به `/api/auth` فرستاده می شود و چون Vercel (و nginx و پراکسی توسعه) `/api` را روی همان `origin` پراکسی می کنند، مرورگر آن را همان سایتی می بیند و CORS لازم نیست. خطاها به صورت `ProblemDetails` می آیند. `PracticeSessionStore` پیام فارسی عمومی نشان می دهد و صفحه ها برای خطای بازگذاری دکمه «تلاش دوباره» دارند.

۱۱. تست

۸۷ تست واحد با Vitest (`npm test`):

- `ApiClient`: مسیر و بدنه هر درخواست با `HttpTestingController`.
- `PracticeSessionStore`: شروع تمرین، ارزیابی فوری در یادگیری، نبودن درخواست در مصاحبه، فاز خودارزیابی، ارسال نهایی، بازگشت بعد از خطا و تمرین خالی.
- `LessonProgress`، `ProgressStore` و `PreferredTechnology`: ثبت، ماندگاری، پاک کردن، `streak` و مقدار نامعتبر `storage`.
- `apiErrorInterceptor`: تبدیل خطاهای HTTP به `ApiError` با پیام فارسی.
- `AuthStore`: ورود، ثبت نام، بازیابی نشست (با و بدون نشانه)، خروج و تغییر رمز.
- `authInterceptor`: افزودن توکن، یک `refresh` مشترک برای چند ۴۰۱ هم زمان، تکرار درخواست و خروج بعد از رد شدن `refresh`.
- `authGuard` و `guestGuard`: انتظار برای بازیابی نشست و هدایت با `redirect`.
- همگام سازی پیشرفت: جابه جایی منبع داده با ورود/خروج و انتقال یک باره داده های محلی.
- صفحه های ورود، ثبت نام و حساب (Signal Forms)، اعتبارسنجی بین فیلدی، خطاهای سرور و پیام های فارسی خطای احراز هویت.
- `ReportDialog` (Signal Form): ارسال دلیل و پیام و جلوگیری از ارسال پیام بیش از ۵۰۰ کاراکتر.
- `LessonExerciseView`: بازنشانی پاسخ قبلی با `linkedSignal` وقتی تمرین عوض می شود.
- `markdown`: رنگ آمیزی، حذف اسکریپت و `onerror`، `escape` زبان های ناشناخته.
- `LessonPage`، `SetupPage` و `LessonPage`: لیست، فیلتر، جست و جوی `debounce` شده با حروف عربی/فارسی، انتخاب زبان، گرفتن درس با زبان جدید و بررسی تمرین. بلوک `@defer` در تست با `DeferBlockBehavior.Manual` دستی رندر می شود، چون `jsdom` از `IntersectionObserver` پشتیبانی نمی کند.
- علاوه بر آن، کل جریان یادگیری و جریان حساب (ثبت نام با انتقال تاریخچه، بازیابی بعد از `refresh` صفحه، تغییر رمز، حذف حساب و بررسی کوکی `httpOnly`) یک بار در مرورگر واقعی (Chromium) روی بک اند واقعی اجرا و تأیید شده است.

۱۲. اجرا و استقرار

۱۲.۱ توسعه

```
npm install
npm start      # http://localhost:4200 (/api به http://localhost:5161 می‌شود)
npm test
npm run build  # خروجی: dist/interviewpal/browser
```

Angular CLI 22 به Node نسخه ۲۲/۲۲ یا بالاتر نیاز دارد (Node 24 پیشنهاد می‌شود). بک‌اند باید روی <http://localhost:5161> در حال اجرا باشد ([proxy.conf.json](#)).

۱۲.۲ Docker

Dockerfile دو مرحله دارد: build با Node 24 و سرو با [nginx.conf.template](#). مسیرهای Angular را به [index.html](#) برمی‌گرداند، فایل‌های hash را یک سال کش می‌کند، [index.html](#) را کش نمی‌کند و [/api](#) را به [API_URL](#) پراکسی می‌کند. [compose.yaml](#) بک‌اند، PostgreSQL و فرانت را کنار هم روی <http://localhost:8080> بالا می‌آورد.

۱۲.۳ Render و Vercel

- فرانت‌اند روی Vercel: preset Angular، دستور `npm run build`، خروجی `dist/interviewpal/browser` و Node 24.x.
- `vercel.json` مسیر `/api/*` را به `https://interviewpal-backend.onrender.com` می‌فرستد و بقیه مسیرها را به `index.html`؛ بنابراین مرورگر فقط یک origin می‌بیند و CORS لازم نیست.
- بک‌اند روی Render با Docker اجرا می‌شود. برای حساب کاربری باید دیتابیس PostgreSQL پایدار و متغیرهای `Auth__JwtKey` (حداقل ۳۲ کاراکتر)، `Database__Provider=Postgres` و `ConnectionStrings__Default` تنظیم شود؛ دیسک پلن رایگان Render پایدار نیست و SQLite آن بعد از هر استقرار پاک می‌شود (جزئیات در مستند بک‌اند). در پلن رایگان پس از ۱۵ دقیقه بی‌کاری می‌خوابد و اولین درخواست بعدی حدود نیم دقیقه طول می‌کشد.

۱۲.۴ مستندات

این سند در [docs/frontend.md](#) نوشته می‌شود و [docs/InterviewPal-Frontend.pdf](#) از روی آن ساخته می‌شود. با هر تغییر مهم، README، همین سند و PDF با هم به‌روز می‌شوند.

۱۳. محدودیت‌ها و گام‌های بعدی

موضوع	وضعیت فعلی	گام بعدی
تأیید ایمیل و فراموشی رمز	ندارد؛ ایمیل تأیید نمی‌شود و رمز فراموش‌شده بازایی نمی‌شود	ارسال ایمیل (لینک تأیید و بازنشانی رمز)
ورود اجتماعی	ندارد	ورود با Google و GitHub

موضوع	وضعیت فعلی	گام بعدی
حالت مصاحبه‌ زمان‌دار	تایمر ساده سمت کلاینت	محاسبه‌ زمان در سرور برای جلوگیری از دستکاری
مرور فاصله‌دار	فقط فهرست سؤال‌های ضعیف	زمان‌بندی مرور بر اساس منحنی فراموشی
پنل ادمین	محتوا با فایل JSON ویرایش می‌شود	پنل مدیریت سؤال و درس و بررسی گزارش‌ها
مصاحبه‌کننده‌ هوشمند	ندارد	پاسخ متنی/صوتی و بازخورد
خواب بک‌اند رایگان	تأخیر اولین درخواست	پلن بدون خواب یا پینگ دوره‌ای